PREMIER UNIVERSITY

Department of Computer Science & Engineering

INTERNSHIP PROJECT REPORT

POS MANAGEMENT SYSTEM

A Laravel-Based Point of Sale and Purchase Management Application

Organization: TORI IT

Submitted By:
Md Imtiaz Uddin
ID No.: **0222220005101132**

Semester	: 8th	Batch	: 42nd
Section	: "B"	Session	: Spring 2026

Submitted To:

Ms. Tashin Hossain
Lecturer, Dept. of CSE
Premier University, Chattogram

Date of Submission**: 2nd November, 2026**	Remarks:

Contents

1. Introduction 1

2. Company Profile 1

3. Organizational Structure 3

4. Technologies Used 3

5. Internship Activities 4

6. Technical Learning 6

7. Problems Observed 7

8. Proposed Solutions 7

9. Reflection on Professional Skills 8

10. Conclusion 8

11. References 9

12. Appendices 9

Appendix A: Project Directory Structure 9

Appendix B: Main Application Routes 10

Appendix C: Main Database Entities 10

Appendix D: Main Development Commands 10

Appendix E: Suggested Screenshots 11

Appendix F: Project Source Code 15

1. Introduction

This report presents the work completed during my internship through the development of a web-based Point of Sale (POS) Management System. The application was developed using the Laravel framework and was designed to reproduce selected features of the POS360 web application.

The main purpose of the system is to support common retail and business operations, including dashboard monitoring, purchase data management, purchase records, purchase orders, customer information, document uploads, and point-of-sale activities. The project provided practical experience in web application development, database integration, Model-View-Controller architecture, form validation, file handling, route management, user-interface implementation, debugging, and dependency management.

The system was developed independently as a Laravel application. The reference system was used to understand the required layout, workflow, and functionality rather than to modify the original application.

2. Company Profile

2.1 Organization Overview

TORI IT is a software development firm that provides information technology solutions to clients in both domestic and international markets. The company began its operations in 2017 and obtained its official registration and license in 2019. Since its establishment, TORI IT has focused on providing technology-based services to meet the growing digital needs of businesses and individuals.

The company operates in four major service areas: mobile application development, website development, software development, and digital marketing. Through these services, TORI IT aims to help businesses improve their digital presence, streamline their operations, and adopt modern technological solutions.

By serving clients both within Bangladesh and abroad, TORI IT contributes to the development of digital services and supports businesses in adapting to the changing technological environment.

2.2 Services and Activities

TORI IT provides four major categories of services:

1. **Mobile Application Development:** Developing mobile applications to meet the specific requirements of businesses and users.
2. **Website Development:** Designing and developing websites that help organizations establish and maintain their online presence.
3. **Software Development:** Creating software applications and customized digital solutions to support business operations and improve efficiency.
4. **Digital Marketing:** Providing digital marketing services to help businesses promote their products and services, reach target audiences, and strengthen their online presence.

These services enable the company to support a wide range of clients with different technological and business requirements in both local and international markets.

2.3 Internship Environment

During my internship at TORI IT, I worked on the development of a web-based Point of Sale (POS) Management System using the Laravel framework. The project provided practical experience in developing a database-driven web application and implementing business-related functionalities.

The internship environment allowed me to apply my academic knowledge to a practical software development project. Through this work, I gained experience in Laravel, PHP, MySQL, Blade templates, Bootstrap, JavaScript, database integration, form validation, file handling, and debugging.

The internship also helped me understand how software development technologies can be used to build business applications that support operational activities and improve the management of information.

3. Organizational Structure

As a software development firm, TORI IT's activities involve different functions related to project planning, software development, design, implementation, and client service. Its major service areas include mobile application development, website development, software development, and digital marketing.

These functions contribute to delivering technology-based solutions according to client requirements. Effective coordination among the relevant functions is important for managing projects, maintaining service quality, meeting deadlines, and ensuring client satisfaction.

The organization operates through teams or departments that work together to plan, develop, test, and deliver software solutions. A typical structure includes management, project management, software development, database administration, quality assurance, technical support, and administration.

The software development process generally follows these stages:

1. Requirement collection
2. System analysis
3. Interface and database design
4. Backend development
5. Frontend development
6. Testing and debugging
7. Deployment and maintenance

My project was primarily related to backend implementation, frontend interface development, database integration, and testing.

4. Technologies Used

Technology	**Purpose**
Laravel	PHP framework for routing, controllers, validation, storage, and application structure.
PHP	Server-side programming language used for application logic.

MySQL	Relational database used to store application records.
Blade	Laravel template engine used to create dynamic views.
HTML/CSS	Page structure, styling, forms, tables, and layout.
Bootstrap	Responsive interface components and layout utilities.
JavaScript	Client-side interactions, filtering, printing, and controls.
Eloquent ORM	Database operations through Laravel models.
Composer	PHP dependency management.
Node.js/npm/Vite	Frontend dependency management and asset development/building.
Git/GitHub	Source-control and project collaboration tools.
XAMPP	Local PHP and MySQL development environment.

5. Internship Activities

5.1 Understanding Requirements

I studied the required business functions and analyzed the reference POS system, including the dashboard, navigation, purchase workflows, purchase orders, customer information forms, search and filtering, file uploads, data tables, and export functions.

5.2 Designing the Application Structure

The application was organized using Laravel MVC architecture. Routes connect browser requests to controllers, controllers process application logic, models manage database records, and Blade views display the results.

5.3 Dashboard Development

The dashboard provides an overview of the system and is intended to display sales activity, purchase activity, product and customer information, recent purchases, quick actions, and system status.

5.4 Purchase Data Management

The purchase data module supports creating, viewing, updating, and deleting customer or purchase-related information. The form includes name, gender, address, telephone, email, occupation, dates, and front and back NID documents.

5.5 File Uploads

The system supports NID document uploads in JPG, JPEG, PNG, and PDF formats. Laravel public storage is used to save the files, with validation for file type and size.

5.6 Purchase Management

The purchase management section provides interfaces for viewing, adding, showing, editing, updating, and deleting purchase records.

5.7 Purchase Orders

The purchase order module includes listing, creation, viewing, editing, updating, deletion, CSV export, and Excel export. The interface is designed to support filtering by supplier, location, status, shipping status, and date range.

5.8 POS Sales Interface

The POS interface provides the main sales screen with customer selection, product search, category and brand filtering, product listing, cart management, payment options, and transaction controls.

5.9 Testing and Debugging

Testing involved checking routes, forms, database operations, validation messages, file uploads, navigation, export functions, and local development environment compatibility.

6. Technical Learning

6.1 MVC Architecture

Models represent database entities, views display information, controllers process requests, and routes connect URLs to controller actions.

6.2 Laravel Routing

I learned to define GET, POST, PUT, and DELETE routes and organize related routes with prefixes and groups.

6.3 Database Migrations

Migrations define database tables and columns through source code, making database setup repeatable.

6.4 Eloquent ORM

Eloquent supports create, read, update, delete, sorting, pagination, and individual record retrieval.

6.5 Form Validation

Validation checks required fields, emails, dates, string lengths, file types, and file sizes.

6.6 File Storage

Uploaded documents can be stored through Laravel?s storage system and served through a public storage link.

6.7 Blade Templates

Reusable layouts and partials reduce duplication and provide a consistent interface.

6.8 Dependency Management

Composer and npm manage project dependencies, and framework versions must be compatible with the installed runtime.

7. Problems Observed

7.1 PHP Version Incompatibility

The project requires PHP 8.3 or newer, while some locked dependencies require PHP 8.4.1 or newer. The initial environment used PHP 8.0.30, so Composer could not install the dependencies and Artisan commands failed.

7.2 Dependency Installation Failure

The PHP mismatch prevented the application from completing its setup process.

7.3 Database Configuration

Incorrect database name, credentials, port, or an unavailable MySQL service can prevent migrations and pages from working.

7.4 Missing Test Data

Without sample records, tables and dashboard sections may appear empty during testing.

7.5 File Storage Configuration

Uploaded documents require correct storage configuration and a public storage link.

7.6 Increasing System Scope

A complete POS system involves products, customers, suppliers, inventory, purchases, sales, payments, and reports, which requires careful planning.

7.7 Module Consistency

Similar purchase workflows require consistent naming, validation, routes, forms, and database behavior.

8. Proposed Solutions

- Upgrade the development environment to PHP 8.4.1 or newer and verify it with php -v.
- Use where.exe php on Windows to ensure the correct PHP executable is first in PATH.
- Configure MySQL and verify the .env database settings before running migrations.
- Create database seeders for products, customers, purchases, and purchase orders.
- Run php artisan storage:link for public uploaded files.

- Add feature and unit tests for CRUD operations, validation, uploads, routes, exports, and database behavior.
- Add authentication, authorization, role-based access, secure document access, and audit logs.
- Improve responsive design, error messages, confirmation dialogs, filtering, pagination, and accessibility.
- Expand reporting with sales, purchasing, supplier, customer, inventory, payment, and profit reports.

9. Reflection on Professional Skills

9.1 Technical Skills

I improved my knowledge of PHP, Laravel, MVC architecture, MySQL, Eloquent ORM, Blade templates, validation, file uploads, HTML, CSS, JavaScript, Composer, npm, debugging, and dependency management.

9.2 Problem-Solving Skills

The PHP and Composer compatibility issue required analyzing error messages and identifying the root cause instead of attempting random commands. This improved my ability to troubleshoot development environments.

9.3 Research and Communication

I learned to study an existing interface, convert requirements into an independent application, organize development tasks, document progress, and explain technical problems clearly.

9.4 Professional Responsibility

The project reinforced the importance of protecting sensitive configuration, validating user input, testing changes, writing maintainable code, and keeping dependencies compatible.

10. Conclusion

The POS Management System project provided valuable practical experience in developing a Laravel-based web application. The system includes a dashboard, purchase data management, purchase records, purchase orders, document uploads, a POS interface, and export functionality.

Through this project, I gained experience in backend development, frontend design, database integration, validation, file management, routing, debugging, and dependency management. The PHP compatibility issue demonstrated the importance of verifying runtime versions before installing project dependencies.

With further development, the system can be expanded with authentication, inventory management, supplier management, sales processing, advanced reporting, role-based permissions, and automated tests. Overall, the project connected theoretical knowledge with practical software development and improved my confidence in building database-driven web applications.

11. References

- Laravel Documentation - https://laravel.com/docs
- PHP Manual - https://www.php.net/docs.php
- MySQL Reference Manual - https://dev.mysql.com/doc/
- Bootstrap Documentation - https://getbootstrap.com/docs/
- Vite Documentation - https://vite.dev/guide/
- Composer Documentation - https://getcomposer.org/doc/
- POS360 reference interface used for studying layout and workflow requirements.
- Project source code and README documentation for the POS Management System.
- GitHub Repository. *POS Management System*. Available at: https://github.com/Imtiaz-1132/pos_app

12. Appendices

Appendix A: Project Directory Structure

- app/ - application controllers, models, and providers
- database/ - migrations, factories, and seeders
- resources/views/ - Blade templates and interface pages
- routes/ - web and console route definitions
- public/ - public application assets
- storage/ - logs and uploaded files
- composer.json - PHP dependencies and scripts

- package.json - frontend dependencies and scripts
- README.md - project overview and module documentation

Appendix B: Main Application Routes

The application includes routes for the dashboard, POS interface, purchase data, purchases, purchase orders, CSV export, Excel export, and create/view/edit/update/delete operations.

Appendix C: Main Database Entities

- User
- Purchase
- PurchaseData
- PurchaseOrder

Appendix D: Main Development Commands

cd "D:\intern Project\pos_app"

composer install

php artisan key:generate

php artisan migrate

php artisan storage:link

npm install

npm run build

php artisan serve

For development with Vite, run npm run dev in a second terminal. The application is then available at http://127.0.0.1:8000.

Appendix E: Suggested Screenshots

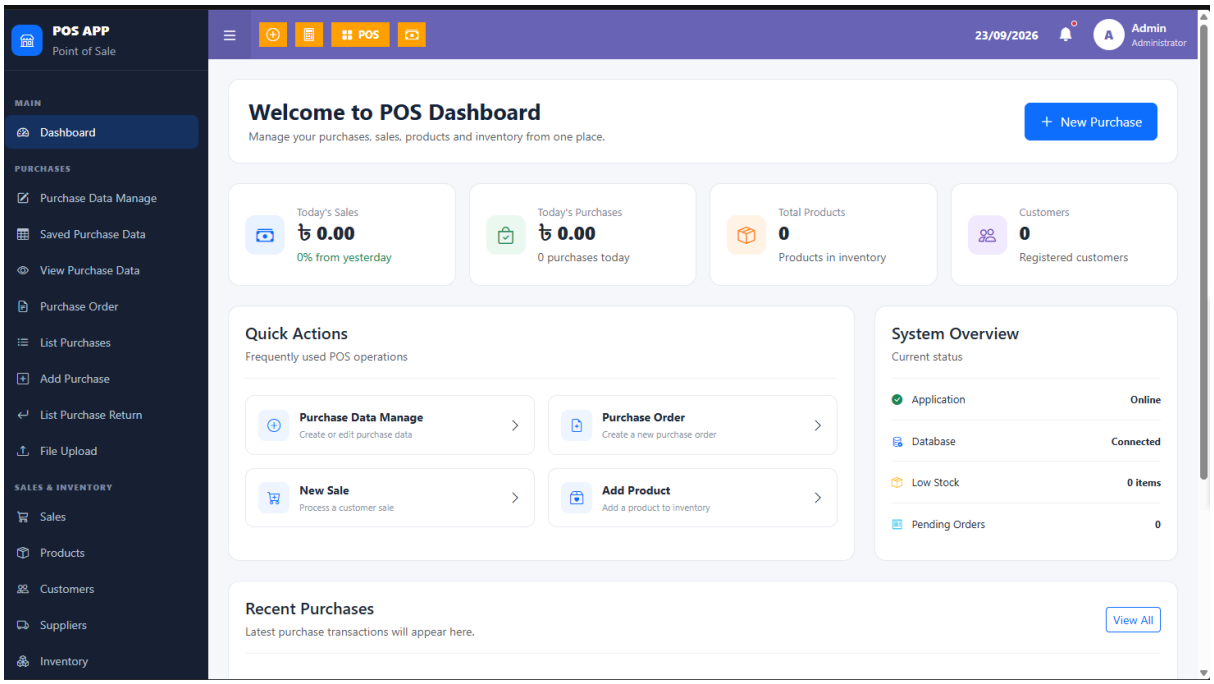


Fig1: Home Dashboard

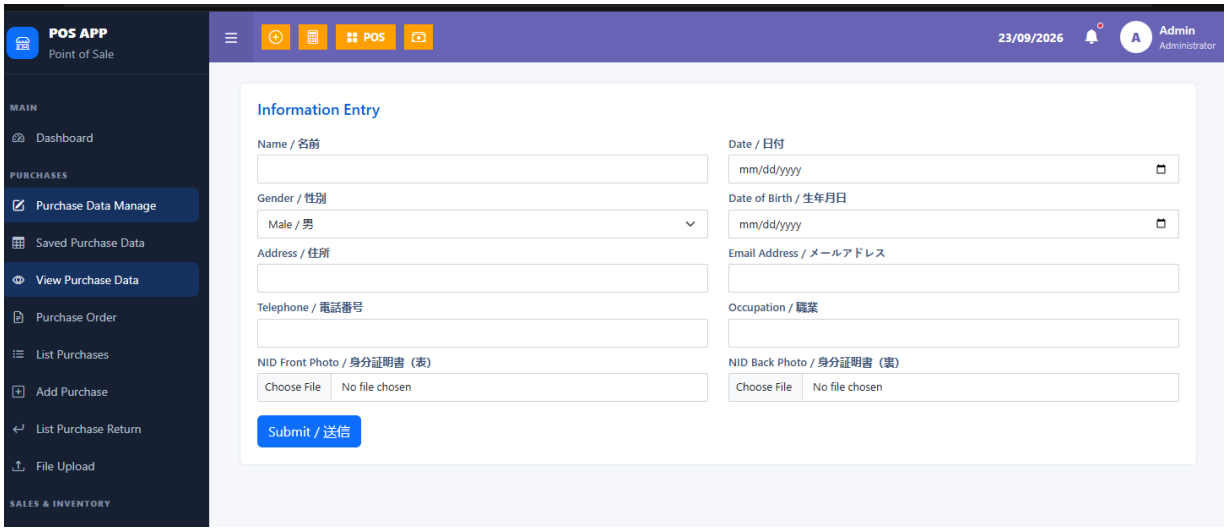


Fig2: Purchase Data Manage

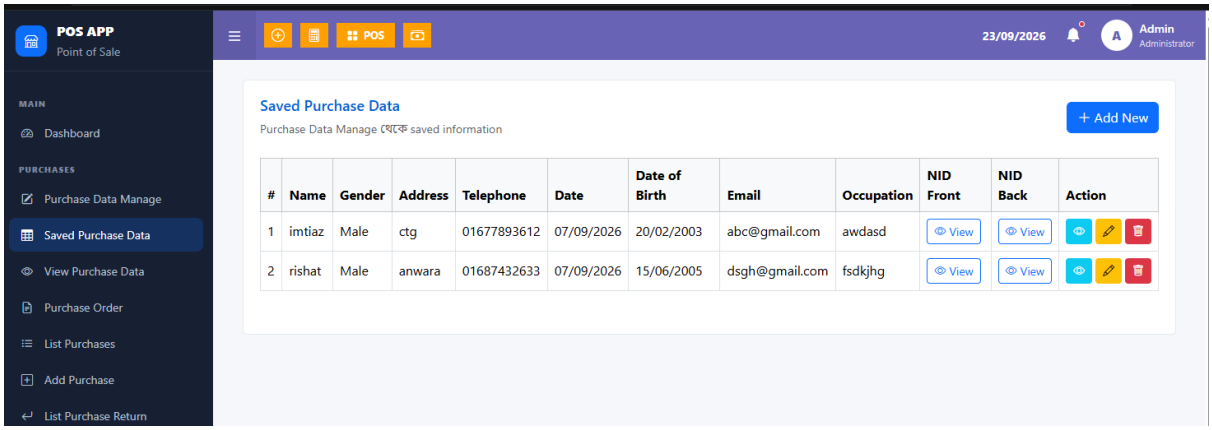


Fig3: Saved Purchase Data

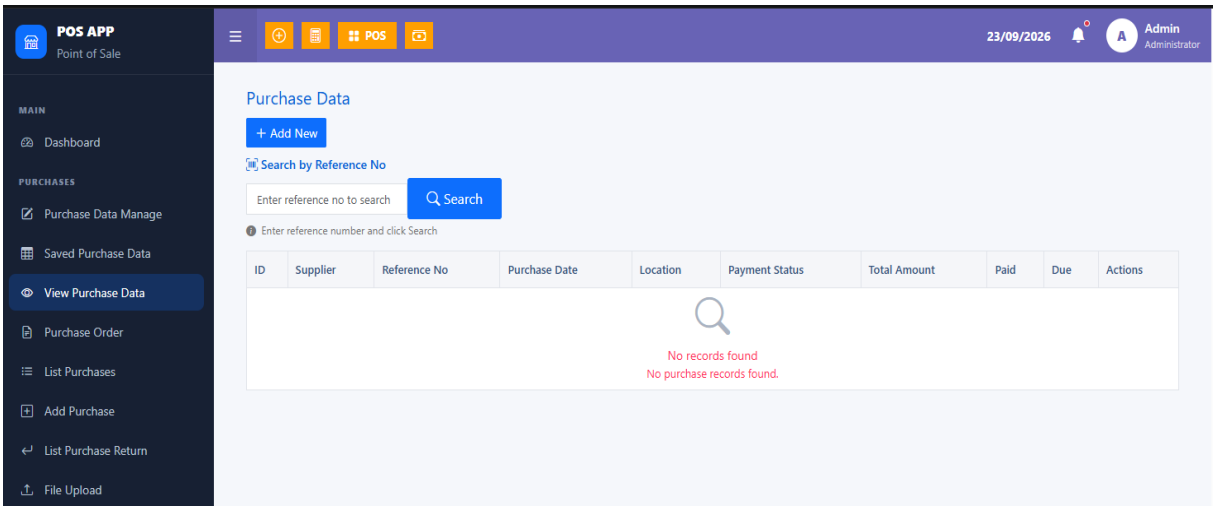


Fig4: View Purchase Data

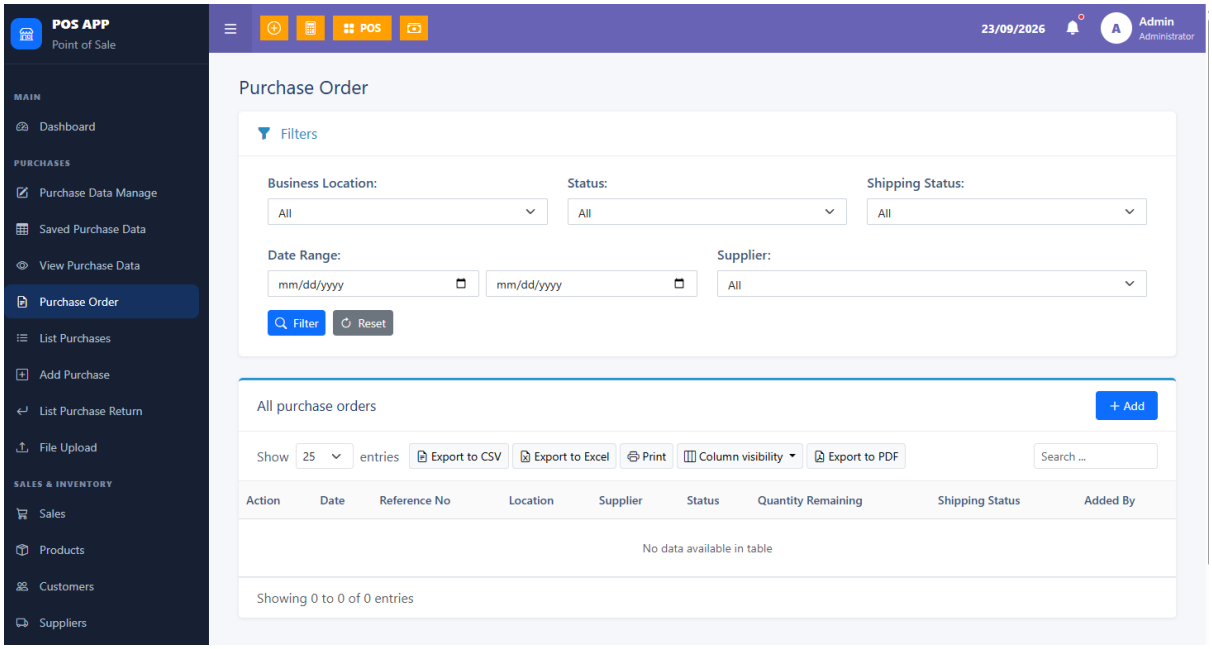


Fig5: Purchase Order

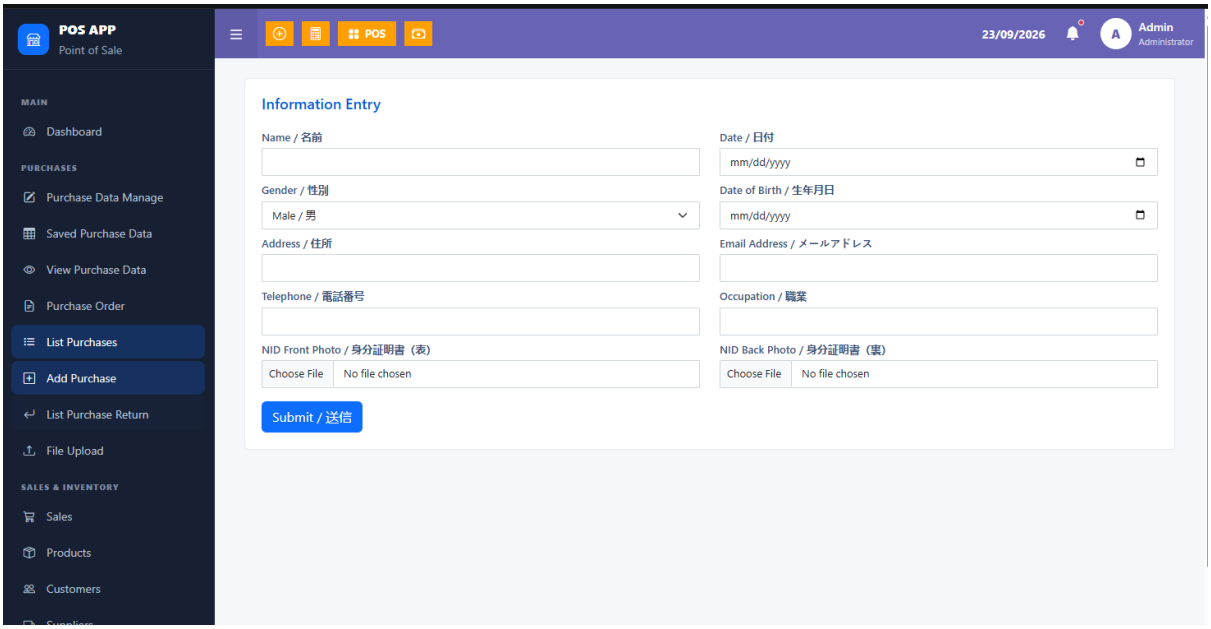


Fig6: Add Purchase

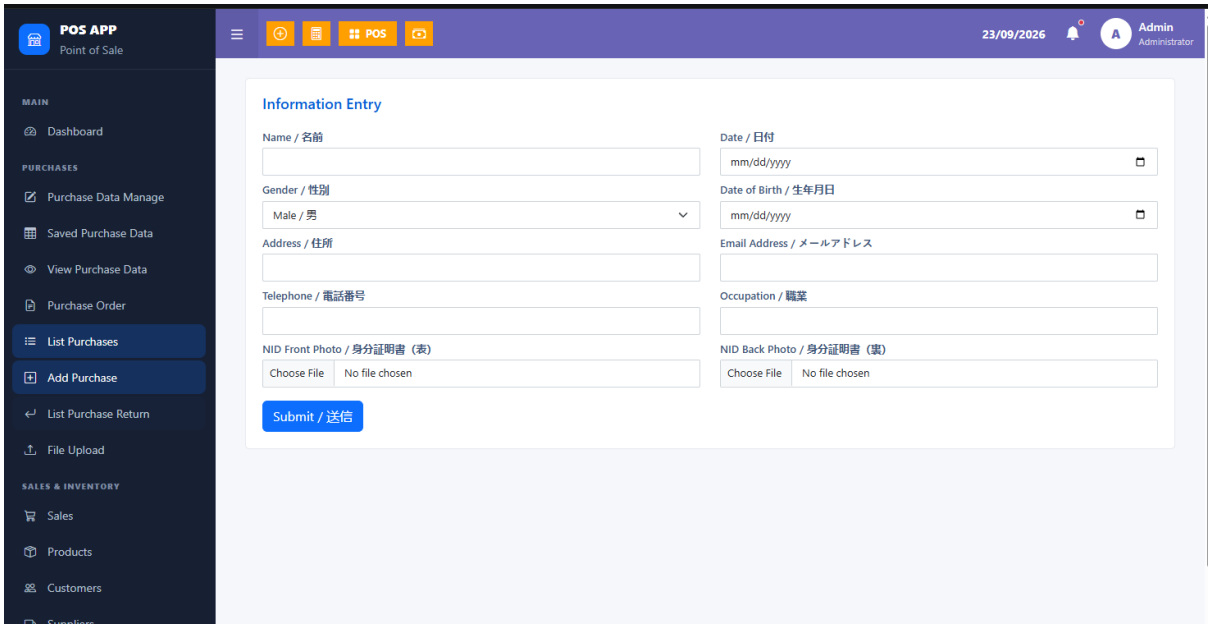


Fig7: POS Home / Sales Interface

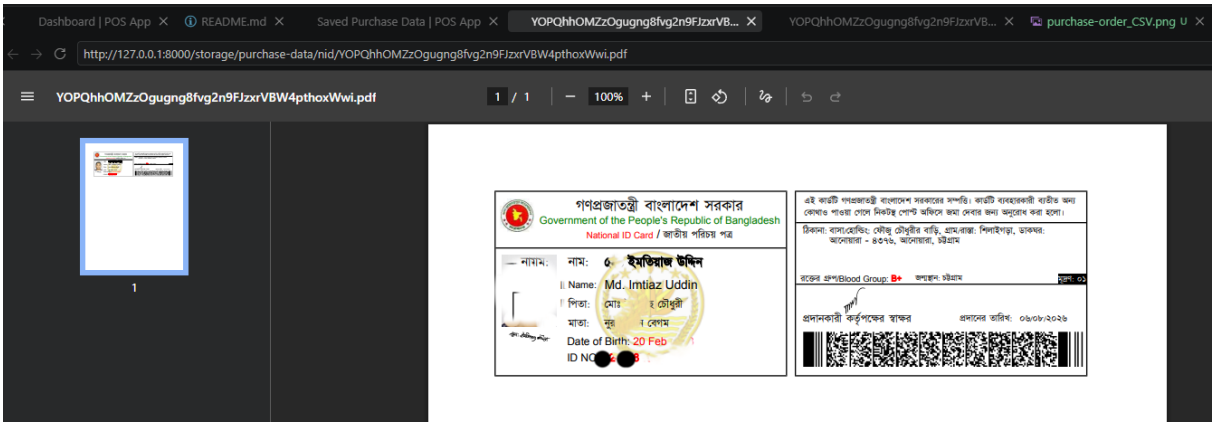


Fig8: NID Document View

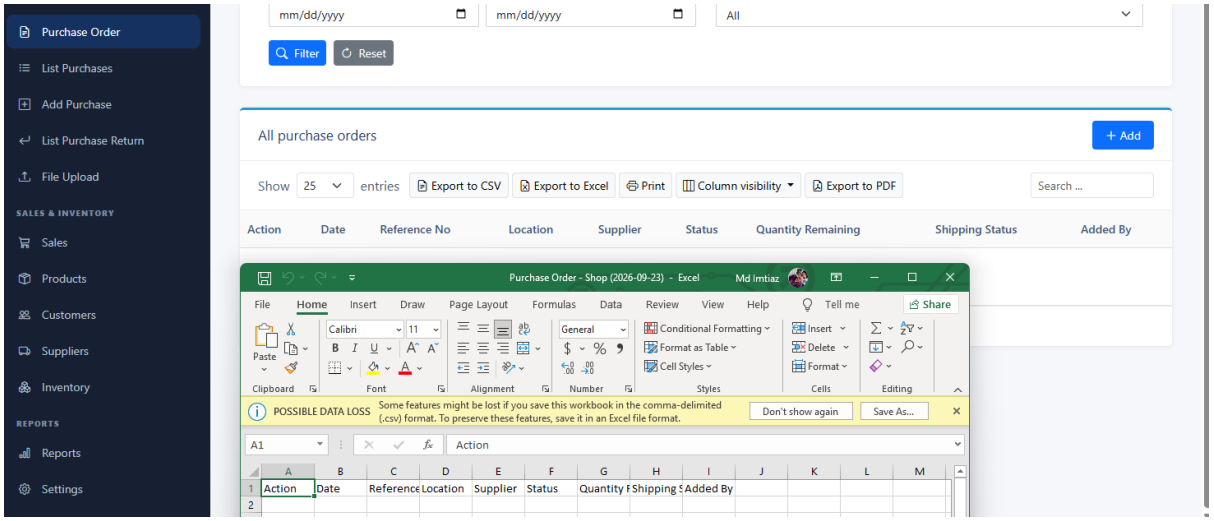


Fig9: Purchase Order CSV Export

Appendix F: Project Source Code

The source code of the POS Management System developed during my internship at TORI IT is available on GitHub. The repository contains the project files and implementation of the Laravel-based application.

Project Name: POS Management System

GitHub Repository: https://github.com/Imtiaz-1132/pos_app

The repository can be accessed to review the source code, application structure, database migrations, controllers, models, Blade templates, and project setup instructions, where available.